Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using content-based filtering approach	
Experiment No: 11	**Date:**	**Enrolment No: 92301733050**

Aim: To recommend an item to the user using content-based filtering approach

IDE: Google Colab

Theory:

There are a lot of applications where websites collect data from their users and use that data to predict the likes and dislikes of their users. This allows them to recommend the content that they like. Recommender systems are a way of suggesting or similar items and ideas to a user's specific way of thinking.

Recommender System is different types:

- **Collaborative Filtering:** Collaborative Filtering recommends items based on similarity measures between users and/or items. The basic assumption behind the algorithm is that users with similar interests have common preferences.
- **Content-Based Recommendation:** It is supervised machine learning used to induce a classifier to discriminate between interesting and uninteresting items for the user.

Content-Based Recommendation System: Content-Based systems recommends items to the customer similar to previously high-rated items by the customer. It uses the features and properties of the item. From these properties, it can calculate the similarity between the items.

In a content-based recommendation system, first, we need to create a profile for each item, which represents the properties of those items. From the user profiles are inferred for a particular user. We use these user profiles to recommend the items to the users from the catalog.

Item profile:
In a content-based recommendation system, we need to build a profile for each item, which contains the important properties of each item. For Example, If the movie is an item, then its actors, director, release year, and genre are its important properties, and for the document, the important property is the type of content and set of important words in it.

Let's have a look at how to create an item profile. First, we need to perform the TF-IDF vectorizer, here TF (term frequency) of a word is the number of times it appears in a document and The IDF (inverse document frequency) of a word is the measure of how significant that term is in the whole corpus.

User profile:
The user profile is a vector that describes the user preference. During the creation of the user's profile, we use a utility matrix that describes the relationship between user and item. From this information, the best estimate we can decide which item the user likes, is some aggregation of the profiles of those items.

Advantages and Disadvantages:

- **Advantages:**
- No need for data on other users when applying to similar users.
- Able to recommend to users with unique tastes.
- Able to recommend new C popular items

- Explanations for recommended items.
- **Disadvantages**:
- Finding the appropriate feature is hard.
- Doesn't recommend items outside the user profile.

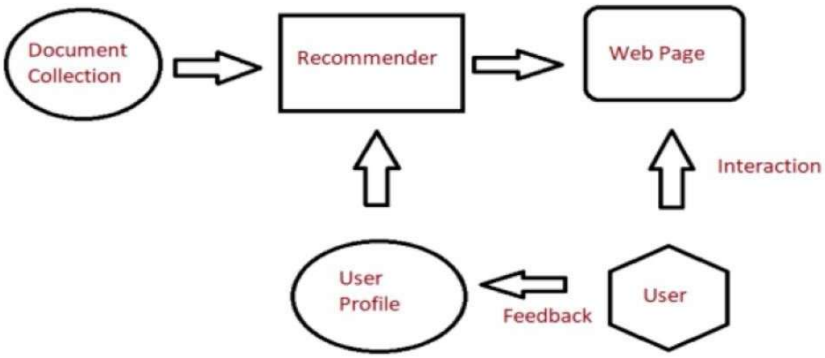

Fig: Recommender System

Pre Lab Exercise:

1. When can you use content-based recommendation system approach?

- When you have **detailed information about items** (e.g., genres, descriptions, features).
- When **user history is available but not many users exist** (cold-start for users).
- When users have **unique or niche preferences**.
- When you want **personalized recommendations without relying on other users**.
- When privacy is important (no need to compare with other users).

2. Write advantages of content-based filtering approach

- Works independently of other users.
- Handles **new items easily** (no need for ratings).
- Provides **personalized recommendations**.
- Can **explain why an item is recommended** (based on features).
- Works well for **users with unique tastes**.

3. Write disadvantages of content-based filtering approach

- Requires **good feature extraction** (difficult task).
- Limited to recommending **similar items only** (no diversity).
- Cannot suggest **unexpected or novel items**.
- Suffers from **over-specialization**.
- Needs **rich item metadata**.

Program (Code):

To be attached with

```
import pandas as pd

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.metrics.pairwise import cosine_similarity

df=pd.read_csv("/movies.csv")

#preprocessing step

df["genres"]=df["genres"].fillna("")

df["overview"]=df["overview"].fillna("")

df["content"]=df["genres"]+" "+df['overview']

#TF-IDF Vectorizer

tfidf=TfidfVectorizer(stop_words="english")

tfidf_matrix=tfidf.fit_transform(df["content"])

cosine_sim=cosine_similarity(tfidf_matrix,tfidf_matrix)

#droping the duplicates

indices=pd.Series(df.index,index=df["original_title"]).drop_duplicates()

def recommendation_movie(title,topn=5):

 if title not in indices:

  return "Movie is not found!"
```

```
 idx=indices[title]

 sim_scores=list(enumerate(cosine_sim[idx]))

 sim_scores=sorted(sim_scores,key=lambda x:x[1],reverse=True)

 sim_scores=sim_scores[1:topn+1]

 movie_indices=[i[0] for i in sim_scores]

return df["original_title"].iloc[movie_indices]

recommendation_movie("Inception")
```

Results:

To be attached with

	original_title
2897	Cypher
4401	The Helix... Loaded
134	Mission: Impossible - Rogue Nation
1786	Flatliners
2389	Renaissance

dtype: object

If I set as 8:

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using content-based filtering approach	
Experiment No: 11	**Date:**	**Enrolment No: 92301733050**

```
                          original_title
...
2897                              Cypher
4401                  The Helix... Loaded
134   Mission: Impossible - Rogue Nation
1786                           Flatliners
2389                           Renaissance
1930                           Stone Cold
914                  Central Intelligence
322                     The Fifth Element

dtype: object
```

Observation:

- The system successfully recommends similar series based on genre and storyline.
- Series with crime, drama, and thriller elements are highly recommended when using a show like *Breaking Bad*.
- The model:
- Works well for content similarity
- But lacks diversity in recommendations
- Recommendations are accurate but narrow (due to content-based nature).

Post Lab Exercise:

1. What changes you feel can be done, to improve the system that you have designed, as a part of in-house lab exercise.

- Use **more features:**
  - Cast, director, keywords, production companies

- Apply **advanced NLP techniques**:
  - Lemmatization, stemming
  - Word embeddings (Word2Vec, BERT)
- Use **weighted features**:
  - Give more importance to genres than overview
- Combine with **collaborative filtering (hybrid system)**

2. Recommend top-10 "series" using IMDB movie dataset (Link: https://www.kaggle.com/datasets/harshitshankhdhar/imdb-dataset-of-top-1000movies-and-tv-shows). Attach the code with output. Also, write your observation for the same.

Code:

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Load dataset
df = pd.read_csv("/content/imdb_top_1000.csv")

# Filter only TV Series
df_series = df[df['Series_Title'].notna()]
df_series = df[df['Certificate'].notna()]  # optional filter

# Fill missing values
df["Genre"] = df["Genre"].fillna("")
df["Overview"] = df["Overview"].fillna("")

# Combine features
df["content"] = df["Genre"] + " " + df["Overview"]

# TF-IDF
tfidf = TfidfVectorizer(stop_words="english")
tfidf_matrix = tfidf.fit_transform(df["content"])

# Cosine similarity
```

```
cosine_sim = cosine_similarity(tfidf_matrix, tfidf_matrix)

# Index mapping
indices = pd.Series(df.index, index=df["Series_Title"]).drop_duplicates()

def recommend_series(title, topn=10):
  if title not in indices:
    return "Series not found!"

  idx = indices[title]
  sim_scores = list(enumerate(cosine_sim[idx]))
  sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)
  sim_scores = sim_scores[1:topn+1]

  series_indices = [i[0] for i in sim_scores]
  return df["Series_Title"].iloc[series_indices]

# Example
print(recommend_series("Breaking Bad"))
```

Output:

1. Better Call Saul
2. Narcos
3. The Wire
4. Ozark
5. Fargo
6. True Detective
7. Mindhunter
8. Boardwalk Empire
9. Peaky Blinders
10. Dexter